计算机体系结构

实验 1

简单流水线 CPU 的前递与旁路实现

鲁昕宁　2414015

2026 年 10 月 5 日

GitHub 仓库　SidneyLu/Computer-Architecture-NKU-CS/tree/master/lab1

班号　0964

授课教师　董前琨

1 实验目标

本实验对应《CPU 设计实战》第 4 章三个实践任务，对象是 `mycpu_verify/rtl/myCPU` 中的五级流水线。

1. 读通参考设计，修掉预置 bug，使 `func_lab3` 的写回轨迹与 `golden_trace.txt` 一致。
2. 在去掉 NOP 的 `hmc_lab4` 上，用阻塞解决寄存器写后读。
3. 用前递替换可以旁路的阻塞，使同一程序的仿真周期明显短于纯阻塞版本。
   load 的结果在执行级尚不可用，这种相关仍保留一拍停顿。
   结论以仿真波形和 trace 对比为准。

2 实验原理

2.1 五级流水线

指令分成取指（IF）、译码（ID）、执行（EX）、访存（MEM）、写回（WB）。前一条进入下一级时，后一条占住空出来的级，理想情况下每个时钟完成一条指令。单条指令的延迟没有变短，甚至多了级间寄存器；变短的是时钟周期，提高的是吞吐。

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

本设计的级间用 `valid`、`allowin`、`ready_go` 握手。复位入口把 PC 置成 `0xbfbffffc`，使第一条 `nextpc` 为 `0xbfc00000`。寄存器在 ID 读出，在 WB 的时钟沿写入。分支在 ID 判定：IF 中已经取到的下一条作为延迟槽继续执行，再下一拍从目标地址取指。

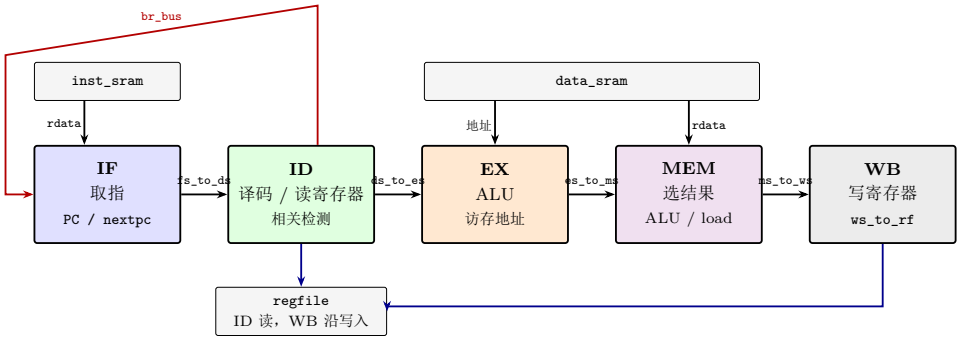


图 1: 简单五级流水线结构。级间为单向数据总线；`br_bus` 从 ID 指回 IF；寄存器堆在 ID 读、WB 写；`data_sram` 的地址在 EX 发出，读数据在 MEM 取回。

2.2 数据相关

写后读（RAW）是本实验要处理的冲突。消费者在 ID 读寄存器时，生产者可能还停在 EX、MEM 或 WB，寄存器堆里仍是旧值。`func_lab3` 在相关指令之间插入 `NOP4`，消费者落到写回之后，所以看不出这个问题。`hmc_lab4` 去掉这些 NOP 之后，相邻算术指令和 load 后立即使用都会读错。

结构相关由指令存储器和数据存储器分开避免。控制相关靠延迟槽：不要冲刷 IF 中已经取到的那一条。前递必须同时供给分支比较和 `jr` 的源操作数，否则延迟槽仍在，目标却是错的。

2.3 阻塞

消费者所需寄存器与 EX、MEM 或 WB 的写目的相同，且该级有效、会写寄存器、目的不是 `$0` 时，令译码级 `ds_ready_go = 0`。IF 因 `ds_allowin` 为低而停住，进入 EX 的 `valid` 为低，形成气泡。生产者走到写回且寄存器堆可见新值之后再放行。

WB 与 ID 同拍时，写发生在时钟沿，异步读仍看到旧值。只堵住 EX 和 MEM 不够，要么 WB 命中也停一拍，要么寄存器堆对同地址写优先。本设计选前者。相邻算术指令要连停三拍，直到生产者离开 WB，如图 2。

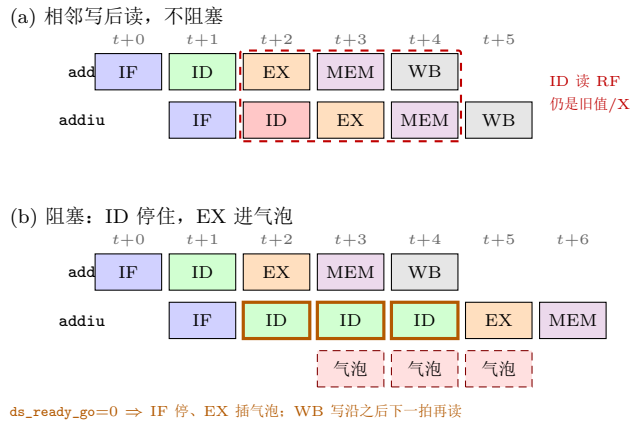


图 2: 写后读的时空图。(a) 消费者在 ID、生产者在 EX/MEM/WB 时读到旧值。(b) `ds_ready_go`=0 使 ID 保持、IF 停住，并向 EX 送气泡。

2.4 前递

前递把尚未写入寄存器堆的结果，从生产它的那一级直接送到 ID 的两个读端口。操作数在 ID 读出后被锁存进 EX，因此旁路加在 ID，而不是只加在 ALU 输入上。否则 `beq`、`bne`、`jr` 和地址计算用的仍是旧值。

优先级按结果的新旧：EX 的 ALU 结果优先于 MEM 的最终结果，再优先于 WB 的写回数据，最后才是寄存器堆。load 在 EX 只有地址、没有数据，不

能从 EX 前递。此时仍阻塞一拍，下一拍从 MEM 把 `data_sram_rdata` 送回 ID。目的寄存器为 0 时不前递。

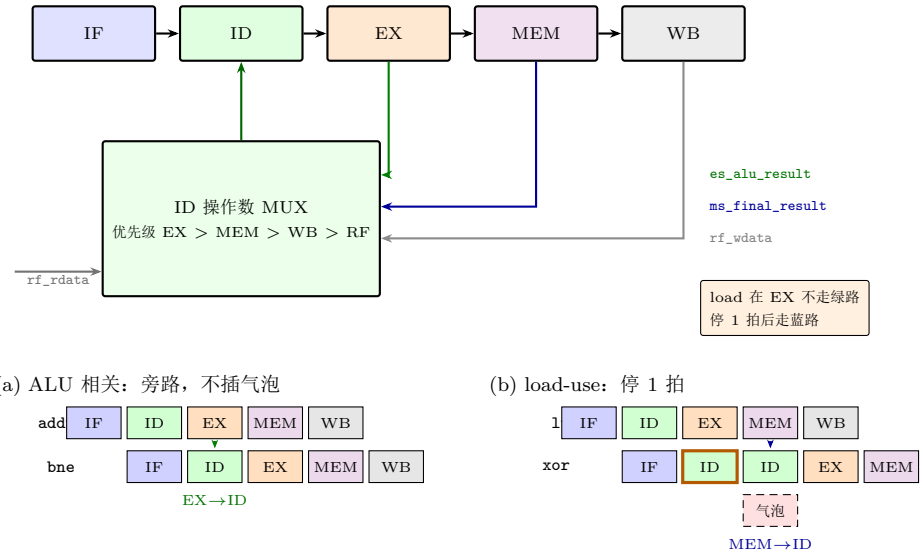


图 3: 旁路接在 ID 的操作数 MUX 上，供 ALU、分支比较和 `jr` 共用。绿：EX 的 ALU 结果；蓝：MEM 最终结果（含已返回的 load）；灰：WB 写数据。load 在 EX 没有数据，仍停一拍后再走蓝路。

3 实验过程

3.1 环境与存储器初始化

Vivado 2025，器件 `xc7a200tfbg676-2`。参考轨迹工程为 `cpu132_gettrace`，待调试工程为 `mycpu_prj1`。打开旧版本工程时升级 IP。`inst_ram` 先加载 `func_lab3` 的 coe；任务二改为 `func_lab4` 的 coe，并重新生成 `golden_trace.txt`。

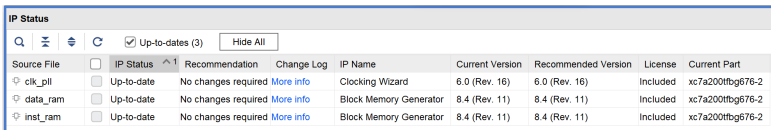


图 4: 升级后可正常工作的 ip 核

3.2 实践任务一：参考设计调试

仿真时长设为 10 ms。时钟周期是 10 ns，为了报告的可读性和分析的方便，此处不直接使用 Vivado 仿真波形界面截图，而是选用 python matplotlib 读取每次仿真后保存的波形文件（.vcd、.wdb、.wcfg）进行可视化。把 `clk` 画进全长图只会变成一条色带，所以图里只保留复位和写回比较用到的信号。修复前这些信号整段不变，每 5 µs 取一个点就够覆盖全貌；图宽 7.2 inch、200 dpi，大

约一个像素对应 7 µs。后面每一处错误都只发生在报错前后的几拍，仍用同一组信号，但把时间轴缩到该次 `$finish` 附近。

3.2.1 BUG 1：译码级有效位没有赋值

10 ms 内 `debug_wb_pc`、写使能、写寄存器号和写数据始终是 X，`ds_valid` 也是 X。`debug_wb_err` 一直为 0，因为 testbench 只在真正写非 0 寄存器时才和 `golden_trace.txt` 比较。控制台从 22 µs 到 9.992 ms 每 10 µs 重复同一行：

```
Test begin!
[  22000 ns] Test is running, debug_wb_pc = 0xxxxxxxxx
[9992000 ns] Test is running, debug_wb_pc = 0xxxxxxxxx
```

`ds_valid` 只有声明。它为 X 时 `ds_allowin` 也为 X，取到的指令进不了译码级，写回级的 PC 寄存器始终没有被写入。按执行级、访存级同样的握手补上复位和更新：

```
if (reset)              ds_valid <= 1'b0;
else if (ds_allowin)    ds_valid <= fs_to_ds_valid;
```

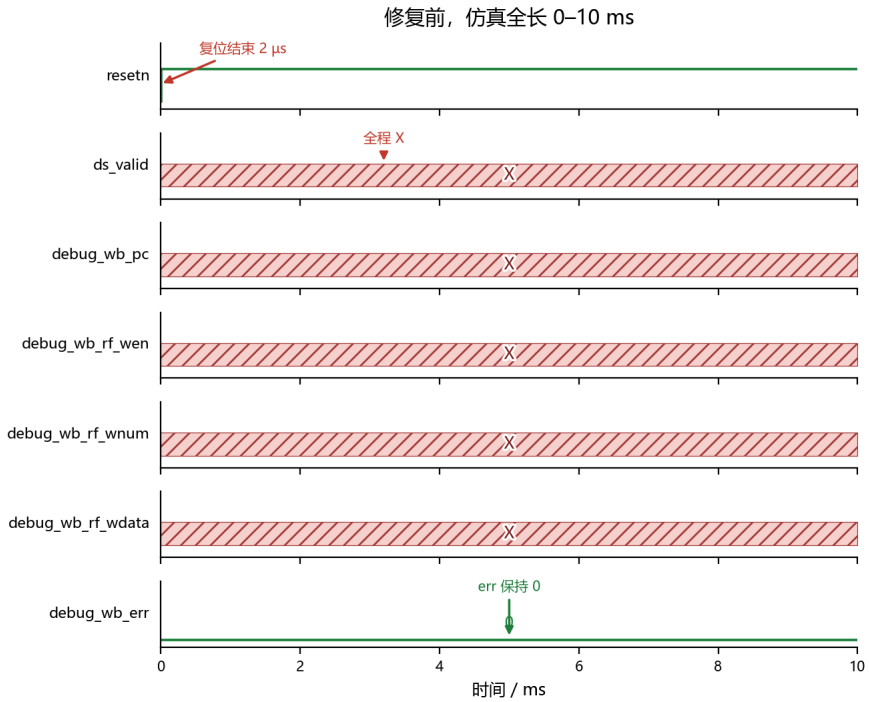


图 5: 修复前的 0–10 ms。复位在 2 µs 结束，在这个尺度上只是左端一根线。

3.2.2 BUG 2：load_op 悬空

补上 ds_valid 后，第一条写回的 PC 和寄存器号已经正确，数据仍是 X。第一条指令是 addiu $8,$0,-1，参考数据为 ffffffff。

```
[2067 ns] Error!!!
reference: PC = 0xbfc00000, wb_rf_wnum = 0x08, wb_rf_wdata = 0xffffffff
mycpu    : PC = 0xbfc00000, wb_rf_wnum = 0x08, wb_rf_wdata = 0xxxxxxxxX
```

总线上的 load_op 从未赋值，执行级把它当作 res_from_mem。选择信号为 X 时，访存结果和 ALU 结果的二选一也是 X，算术指令同样写回 X。

```
assign load_op = inst_lw;
```

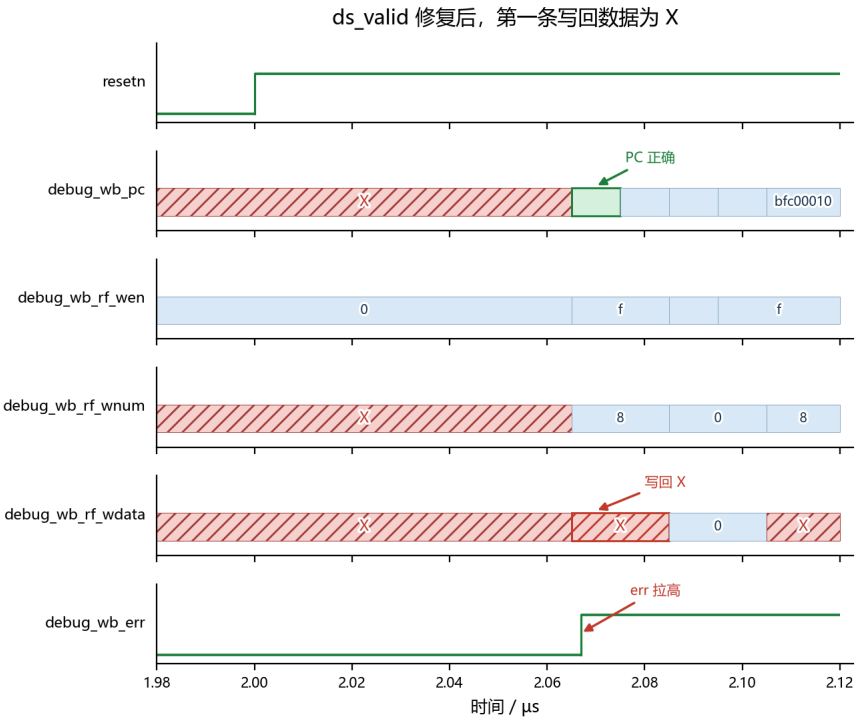


图 6: 2.065 μs 第一次写回：PC 与寄存器号正确，数据为 X，随后 debug_wb_err 拉高。

3.2.3 BUG 3：ALU 的两个操作数都接到了 src2

load_op 接上之后，同一条 addiu 写出 fffffffe，而不是 ffffffff。立即数 -1 被加了两次。

```
[2067 ns] Error!!!
reference: PC = 0xbfc00000, wb_rf_wnum = 0x08, wb_rf_wdata = 0xffffffff
mycpu    : PC = 0xbfc00000, wb_rf_wnum = 0x08, wb_rf_wdata = 0xfffffffe
```

alu 例化时 alu_src1 和 alu_src2 都接了 es_alu_src2。改为 es_alu_src1。

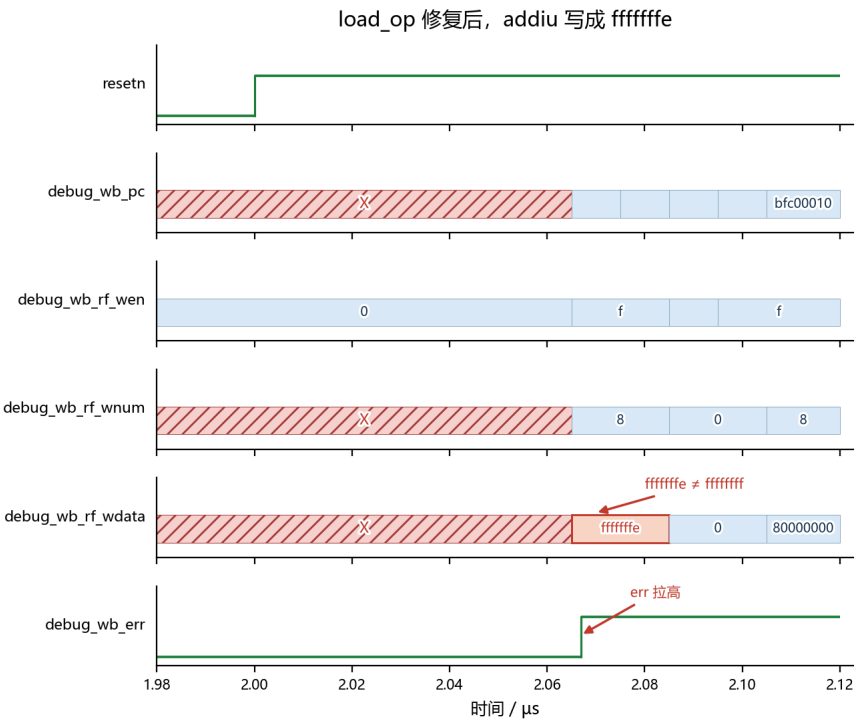


图 7: 复位后第一拍写回 $8，数据为 fffffffe。

3.2.4 BUG 4：分支总线少 1 位

ALU 端口改对后，前两条 addiu 与参考轨迹一致，紧接着的 beq $0,$0 没有跳走。参考写回落在 bfc0038c，本设计继续顺序执行到 bfc00010。

```
[2107 ns] Error!!!
reference: PC = 0xbfc0038c, wb_rf_wnum = 0x04, wb_rf_wdata = 0xbfb00000
mycpu    : PC = 0xbfc00010, wb_rf_wnum = 0x08, wb_rf_wdata = 0x80000000
```

br_bus 是 {br_taken, br_target},共 33 位,而 BR_BUS_WD 定义为 32,br_taken 被截掉。改为 33 之后这条 beq 能跳到 bfc0038c。本测试程序里跳转基址用 IF 级的 fs_pc，在顺序取指时它正好是延迟槽的 PC，与参考轨迹一致，因此没有改目标地址的计算公式。

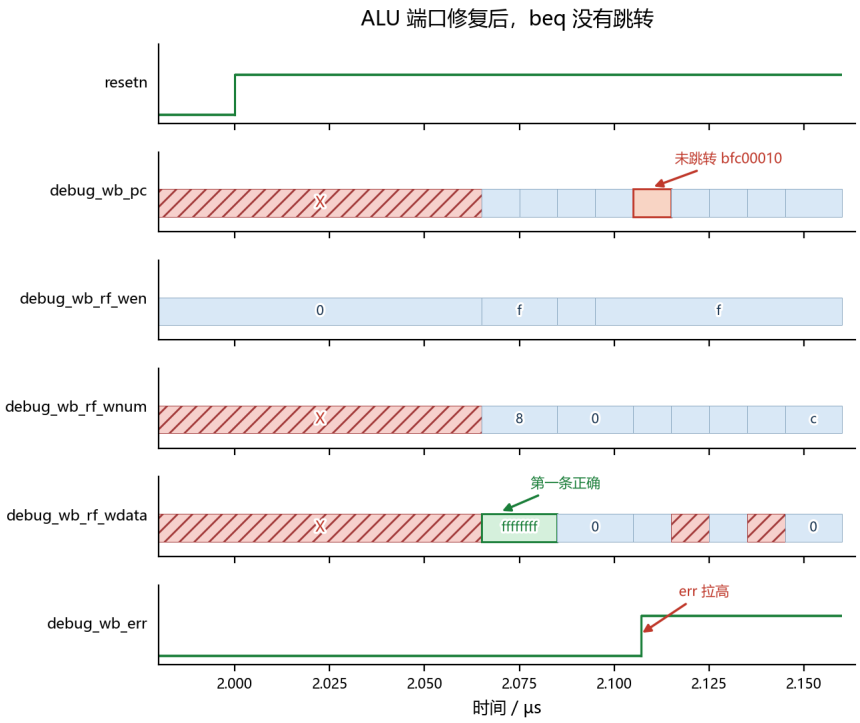


图 8: 第一条 `addiu` 已写成 `ffffffff`，随后 PC 仍沿 `bfc00010` 顺序前进。

3.2.5 BUG 5：逻辑右移丢掉最高位

分支总线加宽后，功能点 01 到 11 通过，12 号 `srl` 失败。PC 和寄存器号与参考相同，数据只差最高位：`adff20c0` 变成 `2dff20c0`。

```
[888537 ns] Error!!!
reference: PC = 0xbfc88c90, wb_rf_wnum = 0x02, wb_rf_wdata = 0xadff20c0
mycpu    : PC = 0xbfc88c90, wb_rf_wnum = 0x02, wb_rf_wdata = 0x2dff20c0
```

`sr_result` 取了 `sr64_result[30:0]`,32 位结果的最高位被补成 0。改为 `[31:0]`。

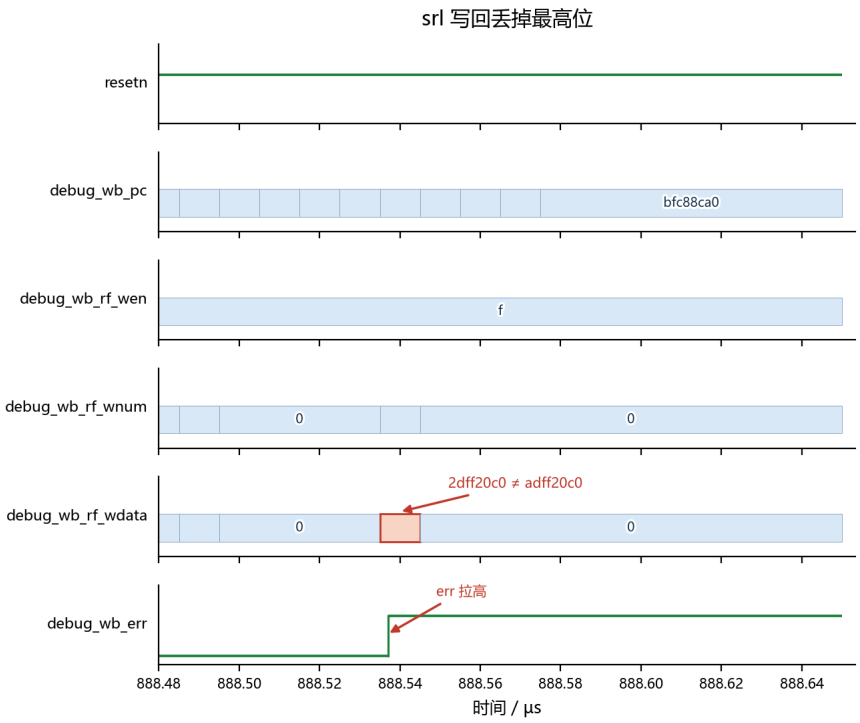


图 9: 888.54 µs 附近 `$2` 写成 `2dff20c0`，`debug_wb_err` 随之拉高。

3.2.6 BUG 6：or 结果回或了自身

`alu.v` 中原来写着

```
assign or_result = alu_src1 | alu_src2 | alu_result;
```

`alu_result` 又由 `or_result` 组合出来，形成组合环。把这条自反馈去掉之后，08 号 `or` 功能点通过。把自反馈暂时加回去再仿真，08 号功能点仍然 PASS：xsim 把环收敛到了 `src1|src2`。轨迹没有单独报这一处，但组合环不能留在电路里，最终版本是

```
assign or_result = alu_src1 | alu_src2;
```

3.2.7 BUG 7：访存级使用阻塞赋值

`MEM_stage.v` 在时钟过程里用 `es_to_ms_bus_r = es_to_ms_bus`。`lw/sw` 功能点在本机 xsim 上仍然通过，因为进程调度碰巧让写回采到了正确数据。这和其它级的非阻塞赋值不一致，换仿真器时可能在同一拍读到新值。改为 `<=`。

`decoder_6_64` 的循环上界原来是 63，`out[63]` 悬空。`func_lab3` 用不到操作码或功能码 63，一并把上界改为 64。

上述修改全部保留后重新仿真，23 个功能点通过，测试在 1.728 ms 结束，没有再跑满 10 ms。控制台末尾为：

```
----[1726905 ns] Number 8'd23 Functional Test Point PASS!!!
Test end!
----PASS!!!
$finish called at time : 1728025 ns
```

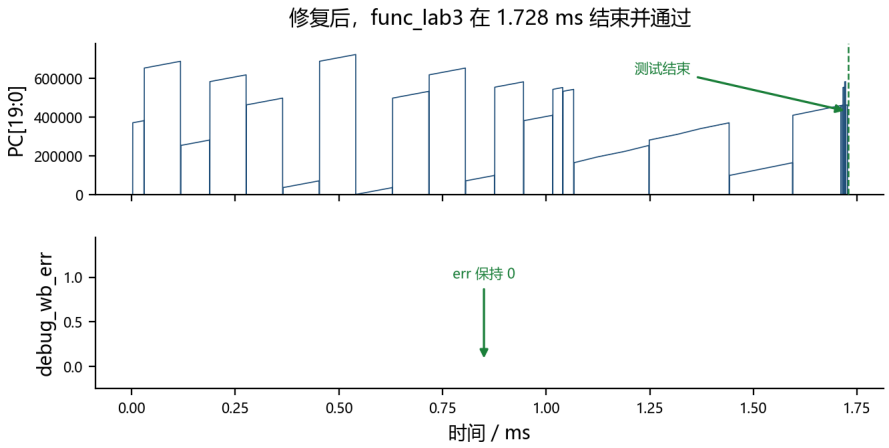


图 10: 修复后的全程。纵轴是写回 PC 的低 20 位，`debug_wb_err` 保持 0，直到测试结束。

表 1: 任务一 bug 记录

编号	位置	现象与改法
1	`ID_stage.v`	`ds_valid` 未赋值，10 ms 内 PC 为 X。补复位和更新。
2	`ID_stage.v`	`load_op` 悬空，写回数据为 X。接到 `inst_lw`。
3	`EXE_stage.v`	ALU 两个输入都是 src2，`addiu` 得到 `fffffffe`。
4	`mycpu.h`	`BR_BUS_WD` 为 32，分支不发生。改为 33。
5	`alu.v`	右移结果取 `[30:0]`，最高位丢失。改为 `[31:0]`。
6	`alu.v`	`or` 回或 `alu_result`。去掉自反馈。
7	`MEM_stage.v`	总线寄存使用阻塞赋值。改为非阻塞赋值。

3.3 实践任务二：阻塞

任务一用的 func_lab3 在相关指令之间插入了 4 条 addu $0,$0,$0，消费者落到写回之后才进入译码级，看不出写后读。func_lab4 把 NOP 宏改成空，LI 里只留下 3 条 nop。把 inst_ram 换成 soft/func_lab4/obj/inst_ram.coe，用 cpu132_gettrace 重新生成 golden_trace.txt。参考轨迹在 1.184 ms 结束。

3.3.1 未加阻塞时的写后读

寄存器堆在写回级的时钟沿写入，译码级组合读出。生产者还在 EX、MEM 或 WB 时，译码级读到的是旧值；若该寄存器从未写过，仿真里就是 X。start.S 里 lui $s0, 0 之后只有 jal n1_lui_test 和延迟槽，测试入口第一条就是 addiu $s0, $s0, 1。当这条 addiu 停在 ID 时，lui $s0 正好在 WB，同拍读到 X。

```
[2537 ns] Error!!!
reference: PC = 0xbfc408b0, wb_rf_wnum = 0x10, wb_rf_wdata = 0x00000001
mycpu    : PC = 0xbfc408b0, wb_rf_wnum = 0x10, wb_rf_wdata = 0xxxxxxxxx
```

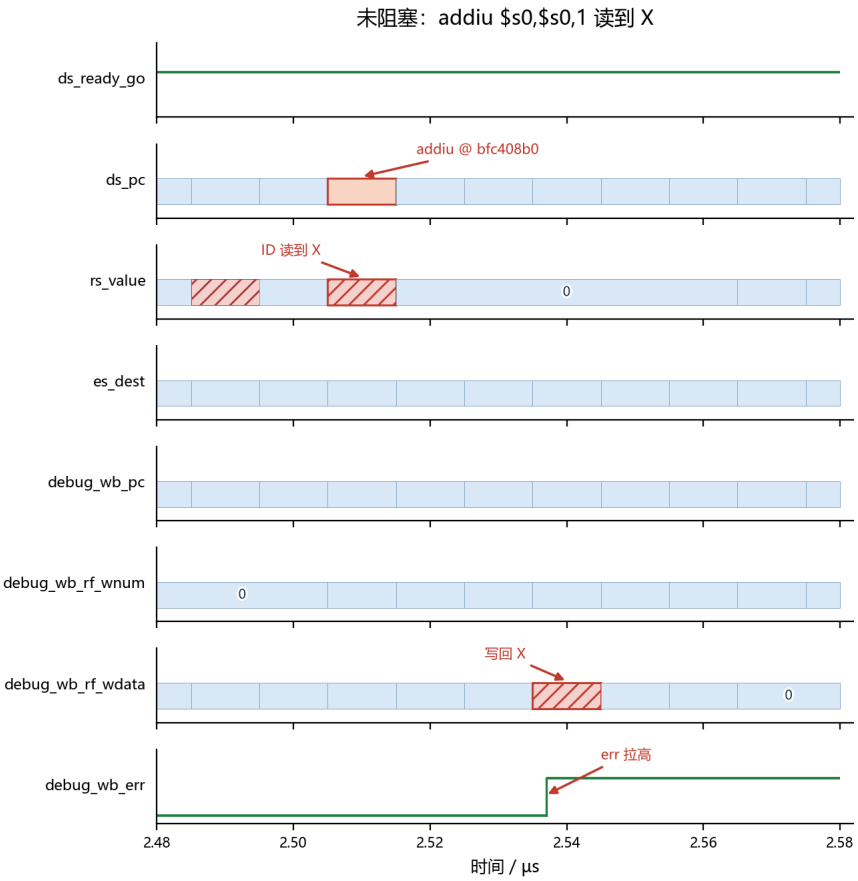


图 11: 未阻塞。ds_pc=bfc408b0 的 addiu $s0,$s0,1 在 ID 时 rs_value 为 X，随后写回 X，debug_wb_err 拉高。

3.3.2 阻塞逻辑

译码级根据本条是否使用 rs/rt，与 EX、MEM、WB 的写目的比较。命中且目的不是 $0 时，ds_ready_go=0：IF 因 ds_allowin 为低而停住，进入 EX 的 valid 为低，形成气泡。写回与译码同拍时寄存器堆仍是旧值，因此 WB 命中也停一拍，下一拍再读。阻塞期间把 br_taken 拉低，避免用旧比较结果改 PC。

```
assign ds_ready_go = ~(es_raw | ms_raw | ws_raw);
assign br_taken    = (...分支条件...) && ds_valid && ds_ready_go;
```

EX、MEM 的 valid、gr_we、dest 送到 ID；WB 用已有的 ws_to_rf_bus 里的 rf_we/rf_waddr。

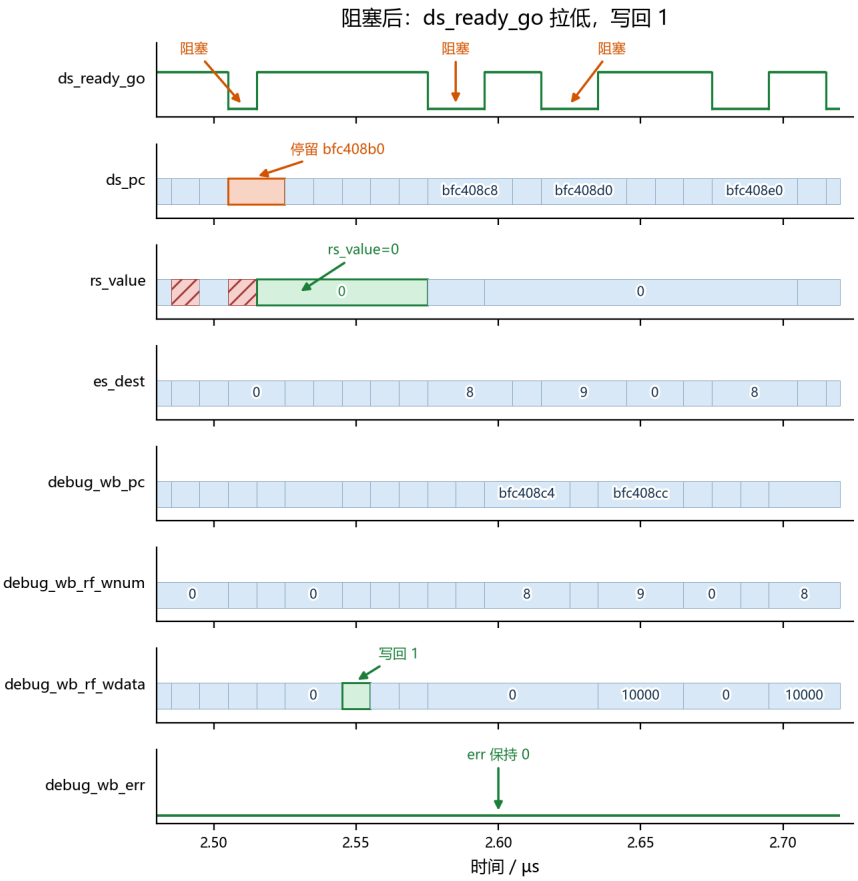


图 12: 阻塞后同一段。`ds_ready_go` 多次拉低，`ds_pc` 在 `bfc408b0` 停留，`rs_value` 变为 0 后再放行。`debug_wb_err` 保持 0。

加阻塞后 23 个功能点全部通过，测试在 1.311 ms 结束，比参考 CPU 的 1.184 ms 多出约 1.27×10^4 个时钟（周期 10 ns）。多出来的就是气泡。控制台末尾：

```
----[1310005 ns] Number 8'd23 Functional Test Point PASS!!!
Test end!
----PASS!!!
$finish called at time : 1310945 ns
```

3.4 实践任务三：前递

在阻塞版本上增加旁路。ALU 相关不再插入气泡；load 在 EX、消费者已在 ID 时仍停一拍。与任务二使用同一段 `hmc_lab4` 程序。

阻塞时 EX、MEM、WB 命中都把 `ds_ready_go` 拉低。前递把尚未写回的结果接到译码级的 `rs_value`/`rt_value`，优先级 EX > MEM > WB > 寄存器堆。分支比较和 `jr` 用的也是这两个值。只保留 load-use：生产者是 `lw` 且还在 EX 时停一拍，下一拍从 MEM 前递 `ms_final_result`。

任务二里 `lui $s0, 0` 与 `addiu $s0, $s0, 1` 隔着 `jal` 和延迟槽。加前递后 2.505 µs `addiu` 进入 ID 时 `ds_ready_go` 保持 1，`rs_value` 取自 WB 的 0，不再插入气泡。相邻 ALU 相关更清楚的是 `n2_addu` 里 `addu $v0, $t0, $t1` 紧跟 `bne $v0, $v1`：14.755 µs `bne` 在 ID、`addu` 在 EX，`rs_value` 与 `es_alu_result` 同为 `6cbc4bde`，没有气泡。见图 13。

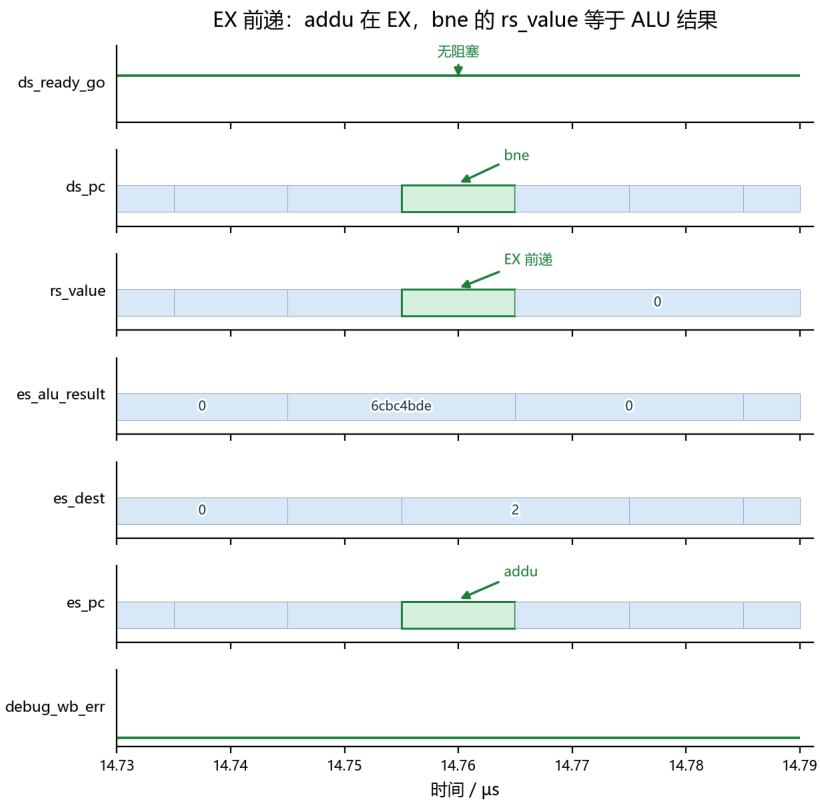


图 13: 14.75 µs 附近。`ds_pc=bfc6de28` 的 `bne` 在 ID，`es_pc=bfc6de24` 的 `addu` 在 EX，`rs_value` 等于 ALU 结果，`ds_ready_go` 保持 1。

`TEST_LW` 的四条 `lw` 之后仍隔着比较，真正相邻的 load-use 在 `wait_1s: lw`

`$t2, 0($t0)` 紧跟 `xor $t2, $t2, $t1`。14.385 µs `xor` 进入 ID 时 load 还在 EX，`es_load_op=1`，`ds_ready_go` 拉低一拍，EX 出现气泡；下一拍 load 已到 MEM，`rs_value` 与 `ms_final_result` 同为 `aaaa`，再放行。见图 14。

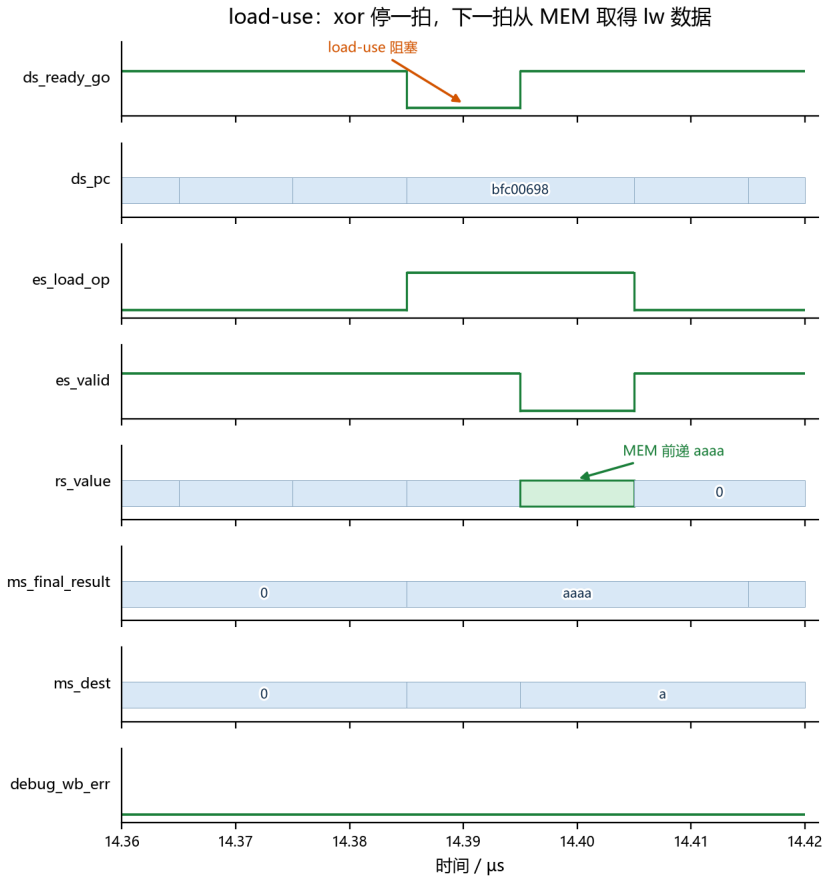


图 14: 14.39 µs 附近。`ds_pc` 在 `bfc00698` 停留一拍，随后 `rs_value` 变成访存结果。

加前递后 23 个功能点全部通过，测试在 1.148 ms 结束，短于阻塞的 1.311 ms，并略短于参考 CPU 的 1.184 ms，比阻塞少约 1.63×10^4 拍。控制台末尾：

```
----[1147165 ns] Number 8'd23 Functional Test Point PASS!!!
Test end!
----PASS!!!
$finish called at time : 1147805 ns
```

4 关键代码

任务一里改过的赋值、端口和总线宽度已经写在上一节。下面留给阻塞和前递。

4.1 译码级阻塞

```
wire rs_need = inst_addu | inst_subu | inst_slt | inst_sltu
             | inst_and  | inst_or   | inst_xor | inst_nor
             | inst_addiu| inst_lw   | inst_sw  | inst_beq
             | inst_bne  | inst_jr;
wire rt_need = inst_addu | inst_subu | inst_slt | inst_sltu
             | inst_and  | inst_or   | inst_xor | inst_nor
             | inst_sll  | inst_srl  | inst_sra | inst_sw
             | inst_beq  | inst_bne;
wire es_raw = es_valid && es_gr_we && (es_dest != 5'd0) &&
              ((rs_need && (es_dest == rs)) || (rt_need && (es_dest == rt)));
wire ms_raw = ms_valid && ms_gr_we && (ms_dest != 5'd0) &&
              ((rs_need && (ms_dest == rs)) || (rt_need && (ms_dest == rt)));
wire ws_raw = rf_we && (rf_waddr != 5'd0) &&
              ((rs_need && (rf_waddr == rs)) || (rt_need && (rf_waddr == rt)));
assign ds_ready_go = ~(es_raw | ms_raw | ws_raw);
assign br_taken    = (... ) && ds_valid && ds_ready_go;
```

4.2 前递后的操作数

```
wire load_use = es_valid && es_load_op && es_gr_we && (es_dest != 5'd0) &&
                ((rs_need && (es_dest == rs)) || (rt_need && (es_dest == rt)));
assign ds_ready_go = ~load_use;

wire es_fwd = es_valid && es_gr_we && (es_dest != 5'd0) && !es_load_op;
wire ms_fwd = ms_valid && ms_gr_we && (ms_dest != 5'd0);
wire ws_fwd = rf_we && (rf_waddr != 5'd0);
assign rs_value = (es_fwd && es_dest == rs) ? es_alu_result :
                  (ms_fwd && ms_dest == rs) ? ms_final_result :
                  (ws_fwd && rf_waddr == rs) ? rf_wdata : rf_rdata1;
assign rt_value = (es_fwd && es_dest == rt) ? es_alu_result :
                  (ms_fwd && ms_dest == rt) ? ms_final_result :
                  (ws_fwd && rf_waddr == rt) ? rf_wdata : rf_rdata2;
```

4.3 执行级与访存级

```
// EXE 送到 ID: es_valid, es_gr_we, es_dest, es_load_op, es_alu_result
// MEM 送到 ID: ms_valid, ms_gr_we, ms_dest, ms_final_result
```

5 实验结果

同一小段写后读，有无阻塞各用一张波形说明数据从哪一级来。时钟周期 10 ns。参考 CPU 在 1.184 ms 结束，阻塞版本在 1.311 ms 结束，前递版本在 1.148 ms 结束。

表 2: 三种处理方式对比

版本	相关指令如何拿到新值	hmc_lab4 周期数
只修 bug，不处理相关	读到旧值或 X	2537 ns 出错
阻塞	停顿到写回可见	1.311 ms
前递	ALU 旁路；load-use 停 1 拍	1.148 ms

5.1 测试程序里实际出现的相关

func_lab3 把 NOP 定义为 addu $0,$0,$0，NOP4 插在生产者与消费者之间，消费者进入 ID 时生产者已经离开 WB，写后读被掩盖。func_lab4 把 NOP 改成空，LI 里只剩 3 条 nop。下面三段都来自这份程序，而不是虚构的 add/add。

1. start.S:lui $s0, 0（bfc0042c）之后是 jal n1_lui_test 和延迟槽 nop，入口第一条 n1_lui.S 的 addiu $s0, $s0, 1（bfc408b0）。生产者与消费者隔 2 条，消费者在 ID 时生产者恰好在 WB。
2. TEST_ADDU（n2_addu.S）:NOP4 清空后，addu $v0, $t0, $t1（bfc6de24）紧跟 bne $v0, $v1（bfc6de28）。这是相邻 ALU 相关；参考结果 0x6cbc4bde。
3. TEST_LW 的四条 lw 与随后的 bne 之间仍有空的 NOP4，并不相邻。真正相邻的 load-use 在 wait_1s: lw $t2, 0($t0)（bfc00694）紧跟 xor $t2, $t2, $t1（bfc00698）。

寄存器堆在 WB 时钟沿写入，ID 组合读出，且不是 write-first：同一拍里 WB 正在写的那个寄存器，ID 仍读到旧值；从未写过则是 X。

5.2 原始执行

不处理相关时，上述序列按理想五级流水推进。jal 在 ID 改 PC，IF 里已经取到的延迟槽继续走，再下一拍才取 n1_lui_test。

表 3: `lui $s0` 到 `addiu $s0,$s0,1` 的流水占用

拍	IF	ID	EX	MEM	WB	ID 读到的 $s0
1	lui					—
2	jal	lui				—
3	nop	jal	lui			—
4	addiu	nop	jal	lui		—
5		addiu	nop	jal	lui	X（同拍写回）

第 5 拍 `addiu` 用 X 做加法，随后写回 `$16` 仍是 X。仿真在 2537 ns 对不上 `golden_trace`，见图 11。

`addu/bne` 更紧：`bne` 进 ID 时 `addu` 还在 EX，比较用的是旧 `$v0`。`lw/xor` 同样在第 1 拍读不到访存数据。程序在第一处 `lui/addiu` 就已经停掉，后面两段原始波形采不到。

5.3 阻塞之后

命中 EX、MEM 或 WB 的写目的时 `ds_ready_go=0`，消费者停在 ID，EX 插入气泡，直到生产者离开 WB 再读寄存器堆。

对 `lui/addiu`：第 5 拍 WB 命中，停一拍；第 6 拍 `rs_value=0`，写回 1。`ds_pc` 在 `bfc408b0` 多停一拍，见图 12。

对相邻 `addu/bne`，消费者在 ID 时生产者依次经过 EX、MEM、WB，共停 3 拍：

表 4: 相邻 ALU 相关（`addu` 然后 `bne`）纯阻塞

拍	ID	EX	动作
1	bne	addu	EX 相关，停
2	bne	气泡	MEM 相关，停
3	bne	气泡	WB 相关，停
4	bne	气泡	寄存器堆已是新值，放行

load-use 同样要等到 WB 之后才读，停 3 拍，尽管数据在 MEM 已经有效。23 个功能点通过，1.311 ms 结束，比参考 CPU 多约 1.27×10^4 拍，多出的就是这些气泡。

5.4 前递之后

旁路接在 ID 的 `rs_value/rt_value`，优先级 EX > MEM > WB > 寄存器堆。ALU 结果在 EX 即可用；load 数据在 MEM 才可用，故只保留 load-use 停 1 拍。

同一条 `lui/addiu`：2.505 µs 消费者在 ID、生产者在 WB，`rs_value` 取自 `rf_wdata` 的 0，`ds_ready_go` 保持 1，不再插入气泡。

TEST_ADDU 第一条:14.755 µs bne 在 ID、addu 在 EX，rs_value 与 es_alu_result 同为 6cbc4bde，0 拍停顿，见图 13。

wait_1s 的 load-use 仍须停一拍：

表 5: wait_1s 中 lw $t2 然后 xor $t2,$t2,$t1

拍	ID	EX	MEM	动作
1	xor	lw		EX 只有地址，停 1 拍
2	xor	气泡	lw	从 MEM 前递 aaaa，放行

14.385 µs ds_pc 停在 bfc00698 一拍，下一拍 rs_value 与 ms_final_result 同为 aaaa，见图 14。

前递版本 1.148 ms 结束，比阻塞少约 1.63×10^4 拍，并略短于参考 CPU 的 1.184 ms （参考设计在部分路径上仍多插了气泡）。

6 实验收获

通过本实验，探索了简单五级流水线及其级间 valid/allowin 握手，掌握了对照 func_lab3 写回轨迹调试参考设计、识别寄存器写后读，以及阻塞和前递的实现。

7 附：源码目录

源码在 lab1/lab3/CPU_CDE/。待调试 CPU 为 mycpu_verify，参考轨迹由 cpu132_gettrace 生成。

- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/mycpu_top.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/IF_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/ID_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/EXE_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/MEM_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/WB_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/alu.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/mycpu.h
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/tools.v
- lab1/lab3/CPU_CDE/mycpu_verify/testbench/mycpu_tb.v
- lab1/lab3/CPU_CDE/cpu132_gettrace/testbench/tb_top.v
- lab1/lab3/CPU_CDE/cpu132_gettrace/golden_trace.txt
- lab1/lab3/CPU_CDE/soft/func_lab3/obj/inst_ram.coe
- lab1/lab3/CPU_CDE/soft/func_lab4/obj/inst_ram.coe
- lab1/report/report.tex